Architecture et théorie des systèmes d’exploitation

Plan du cours

- Introduction aux systèmes d'exploitation
- Architecture et gestion de la mémoire
- Gestion et ordonnancement des processus
- Détection d'erreurs et inter-blocages des processus
- Gestion des ressources

Introduction aux systèmes d'exploitation

- Un système d'exploitation est une interface entre l'utilisateur et la machine physique, il permet d'utiliser les fonctionnalités de l'ordinateur.
- Il fournit une base appelée « machine virtuelle » sur laquelle seront construits les programmes et utilitaires.
- Le but du système d'exploitation consiste à permettre de développer des applications sans se soucier des détails de fonctionnement et de gestion du matériel.
  - Exemple : on peut effectuer la lecture d'un fichier par un simple appel système « read » sans se soucier sur quel type de support il se trouve (Disque, Cle USB, …).

Les fonctions d'un système d'exploitation

- Les systèmes d'exploitation ont comme fonctions principales :
  - Chargement et lancement des programmes
  - Gestion des processeurs, de la mémoire, des périphériques
  - Gestion des processus (programmes en cours d'exécution) et du système de fichiers
  - Détection des erreurs

Structure d’un système d’exploitation

- Le système d’exploitation est structuré en couches : chaque couche utilise les fonctions des couches inférieures. On l’organise généralement en 5 couches :
  - Au plus bas niveau on trouve le noyau, l’interface entre le matériel et le logiciel, il se charge de gérer le CPU, les interruptions, la communication et la synchronisation entre les processus. Il réside entièrement en mémoire.
  - Au second niveau, on trouve le gestionnaire de la mémoire qui se charge du partage de la mémoire entre les processus.

Structure d’un système d’exploitation

- Au troisième niveau, on a le module de gestion des entrées / sorties qui se charge de gérer tous les périphériques (clavier, écran, disques, imprimantes, …).

- Au quatrième niveau, on trouve le gestionnaire de fichiers qui se charge de la gestion de l’espace disque, de la manipulation des fichiers tout en assurant l’intégralité des données, la protection des fichiers.

Structure d’un système d’exploitation

- Au cinquième niveau, on a le module d’allocation des ressources qui se charge d’assurer une bonne utilisation des ressources :
  - Il crée de nouveau processus et leur attribue un niveau de priorité.
  - Il permet à chaque processus d’obtenir les ressources nécessaires dans la limite de temps raisonnables.
  - Il permet d’exclure mutuellement les processus qui demandent une ressource non partageable et d’éviter les situations de blocage.

Les processus

- Un processus est un programme en cours d'exécution.
- On distingue deux sortes de processus :
  - Les processus systèmes qui accomplissent une tâche pour le compte du système (gérer l'imprimante par exemple)
  - Les processus utilisateurs
- Les processus exécutent leur code et demande des services au noyau par l'intermédiaire des appels système (open, read, write, etc.)
- Un processus peut créer un ou plusieurs processus fils qui à leur tour peuvent créer des processus fils dans une structure arborescente.

La mémoire

- Dispositif capable d'enregistrer, de conserver et de restituer des informations.

- On classe les mémoires selon :
  - Caractéristiques : capacité, débit …
  - Type d'accès : séquentiel, direct …

- Mot : unité d'information adressable en mémoire (groupement d'octets)

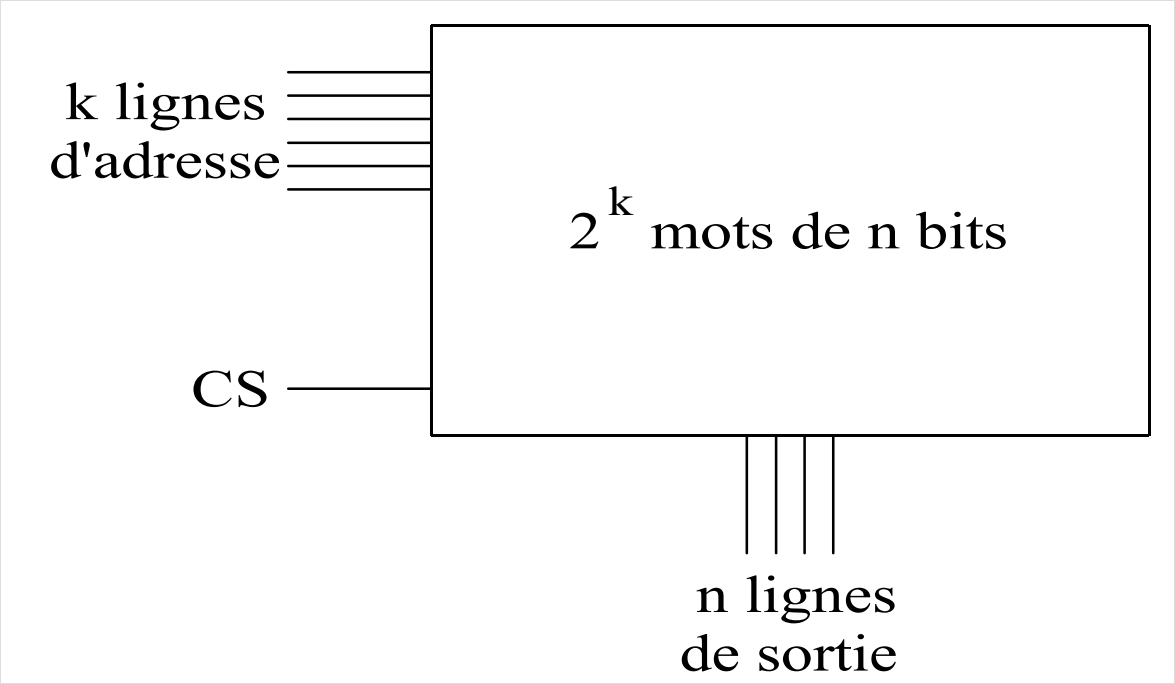
- Adresse : valeur numérique référençant un élément de la mémoire (mot)

Types de mémoire

- 2 grandes familles
- Mémoires non volatiles : ROM (Read Only Memory) dites mémoires mortes
    - Leur contenu est fixe (ou presque ...)
    - Conservé en permanence
- Mémoires volatiles : RAM (Random Access Memory) dites mémoires vives
    - Leur contenu est modifiable
    - Perte des informations hors alimentation électrique
    - Random : à prendre dans le sens « accès sans contraintes » (et non pas aléatoire)

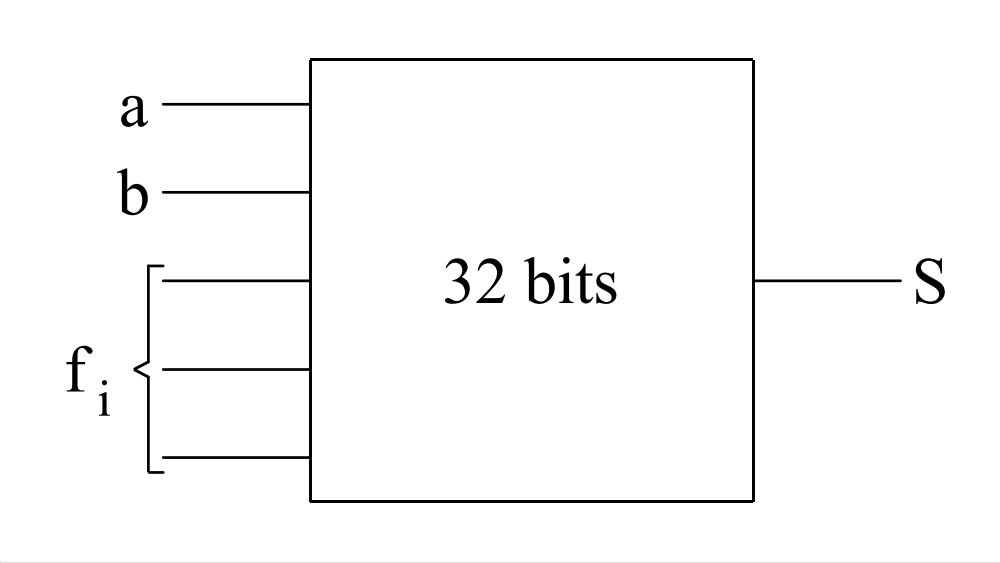
Mémoire morte (ROM)

- Extérieurement une mémoire morte ressemble à un bloc mémoire sans ligne d'entrée ni commande d'écriture. Ce système peut se restreindre à k lignes d'adresse, n lignes de sortie et un signal de sélection du circuit.



Mémoire morte : exemple

- Il est facile d'écrire la table de vérité des huit fonctions par rapport aux quatre combinaisons de a et b. Nous obtenons ainsi 8 x 4 = 32 valeurs possibles. Celles-ci peuvent être mémorisées dans une ROM 32 bits comptant 5 lignes d'adresse et une sortie. Trois bits servent à identifier la fonction fi, les deux autres lignes d'adresse correspondent aux variables a et b.



$$\begin{cases} f_0(a,b) = a \bullet b \\ f_1(a,b) = a \bullet \overline{b} \\ f_2(a,b) = \overline{a \bullet b} \\ f_3(a,b) = a + b \\ f_4(a,b) = a + \overline{b} \\ f_5(a,b) = \overline{a + b} \\ f_6(a,b) = a \oplus b \\ f_7(a,b) = a \oplus \overline{b} \end{cases}$$

Mémoires volatiles (RAM)

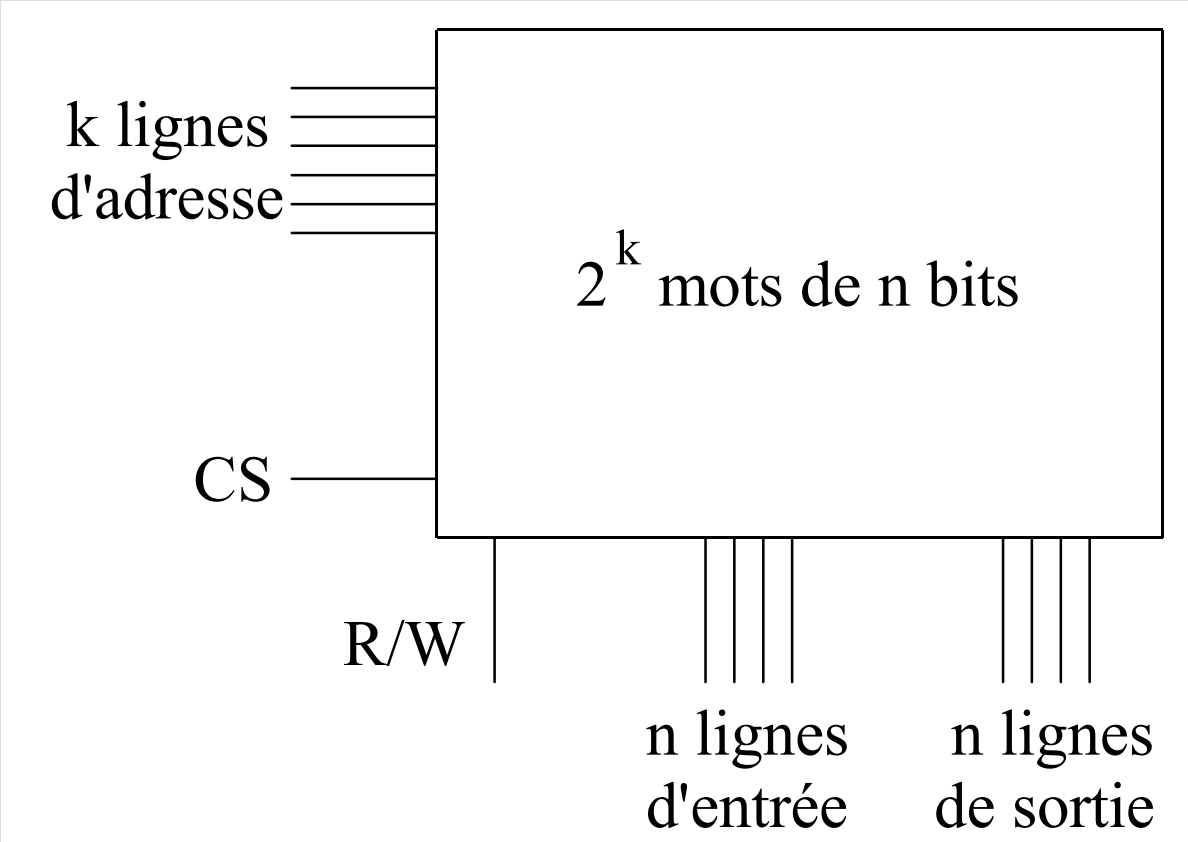
- 2 grands types de RAM

- DRAM : Dynamic RAM
    - Dynamique : nécessite un rafraîchissement périodique de l'information (condensateur)
    - Peu coûteuse

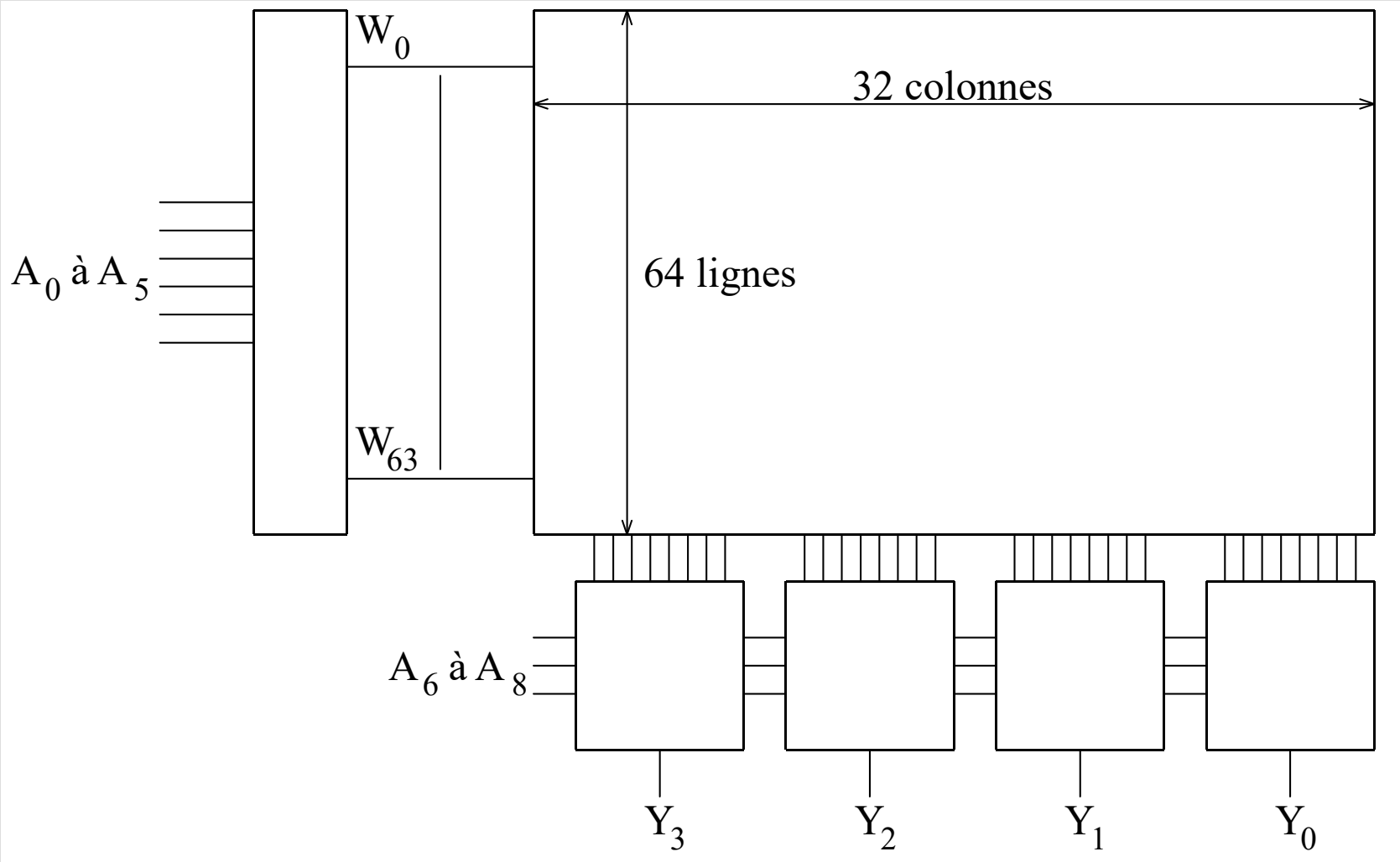
- SRAM : Static RAM
    - Statique : ne nécessite pas de rafraîchissement
    - Beaucoup plus rapide que la DRAM
    - Mais beaucoup plus chère

Mémoire vive : la RAM

RAM : Random Acces Memory



Mémoire : adressage matriciel



Mémoire cache (SRAM)

- Le processeur a besoin d'un débit soutenu en lecture d'instructions et de données pour ne pas devoir attendre sans rien faire.

- Problème

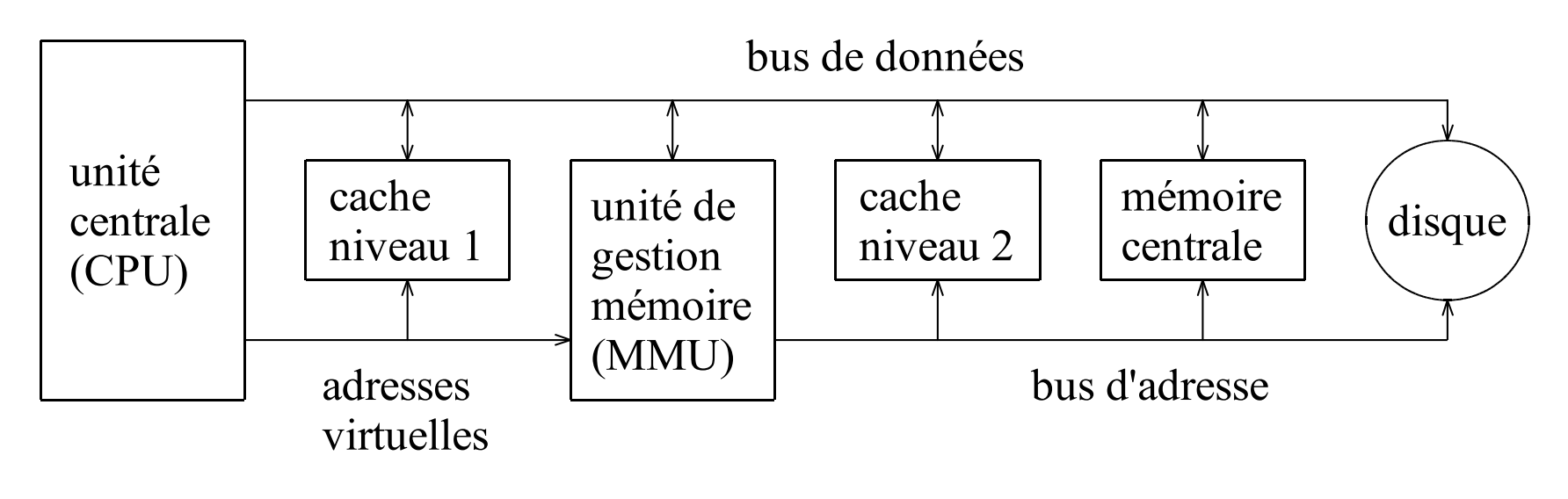
Mémoire centrale qui stocke ces instructions et données est beaucoup trop lente pour assurer ce débit.

- Idée

Utiliser une mémoire très rapide intermédiaire entre la mémoire centrale et le processeur (mémoire cache)

Mémoire cache (SRAM)

Mémoire tampon



Permet de gérer en partie la différence de vitesse entre l'unité centrale et la mémoire centrale

Registres

- Se trouvent intégrés dans le CPU

- Un registre est un mot stockant des informations relatives à une instruction : 64 bits pour une machine Windows 64 bits

- Peu nombreux dans un CPU

- Très rapides (vitesse du CPU)

La mémoire virtuelle

Principe: on divise la mémoire physique en **blocs** ou cadres de taille fixe.

• La zone mémoire (adresses logiques) de chaque processus est divisée en **pages** de la même taille que les blocs.

• Le processeur possède un registre dédié contenant l'adresse (physique) d'une **table de correspondance**.

Mémoire virtuelle

- La table des blocs (table de correspondance) d'un processus indique quels sont les blocs en mémoire. Elle contient une entrée pour chaque bloc de l'espace virtuel du processus (bit de présence en mémoire, adresse physique du début du bloc, ….)

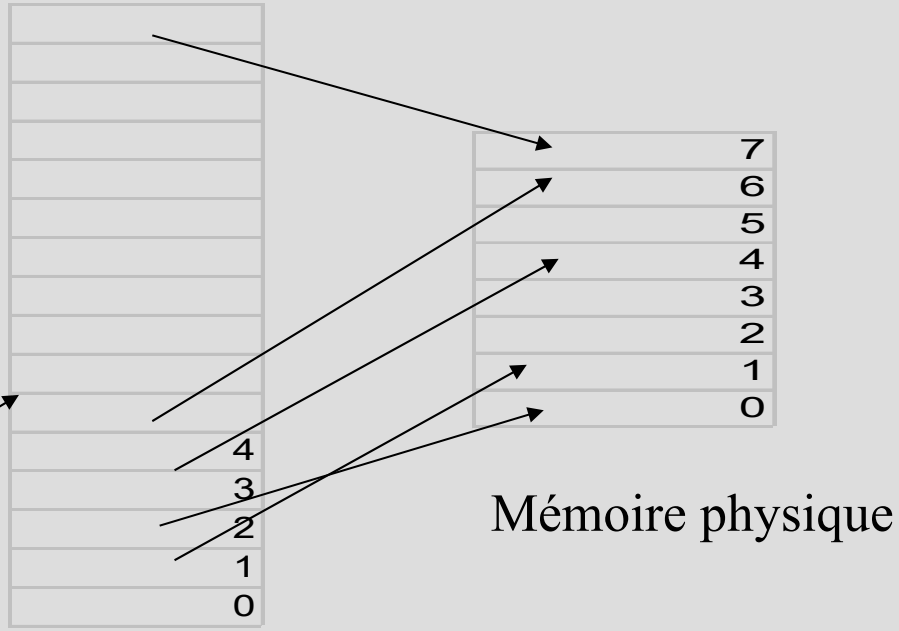
- L'adresse de la table des blocs fait partie du contexte du processus à sauver ou à restaurer lors du changement de contexte.

- Il faut :
  - un mécanisme de conversion rapide des adresses virtuelles en adresses physiques.
  - une politique de remplacement des blocs.

Mémoire virtuelle : la pagination

- La mémoire virtuelle et la mémoire physique sont structurées en pages pour la mémoire virtuelle et en cases pour la mémoire physique.
- La taille d ’une page est fixe est est égale à celle d’une case. Elle varie entre 512 octets et 8 Ko

- Exemple d’un programme de 64 Ko sur une machine de 32 Ko de mémoire physique (avec des pages de 4 ko) avec une instruction d’un octet.



Conversion d’adresses virtuelles

- Les adresses virtuelles référencées par l’instruction en cours d’exécution doivent être converties en adresses physiques
- La correspondance entre les pages et les cases est mémorisée dans une table appelée **table des pages**. Le nombre d ’entrées de cette table est égal au nombre de pages virtuelles
- La table des pages d ’un processus doit être en mémoire centrale lors de l’exécution du processus.
- Chaque entrée de la table des pages est composée de plusieurs champs, notamment :
  - le bit de présence en mémoire
  - le bit de référencement (R)
  - le bit de modification (M)
  - le numéro de la case correspondant à la page
  - son emplacement sur le disque

Conversion d'adresses virtuelles exemple : sur 16 bits

0	**0**	**1**	**0**	0	0	0	0	0	0	0	0	0	1	0	0

Adresse virtuelle

8196

Table des pages

1	0	0	1	**1**
2	1	1	0	**1**
3	0	0	0	**1**
4	1	0	0	**1**
5	0	1	1	**1**
6	0	0	0	**0**
7	0	0	0	**0**
8	0	0	0	**0**
9	1	0	1	**1**
10	0	0	0	**0**
12	1	1	1	**1**
13	0	0	0	**0**
14	0	0	0	**0**
15	0	0	0	**0**
16	0	0	0	**0**

110

Bit de présence

Adresse physique

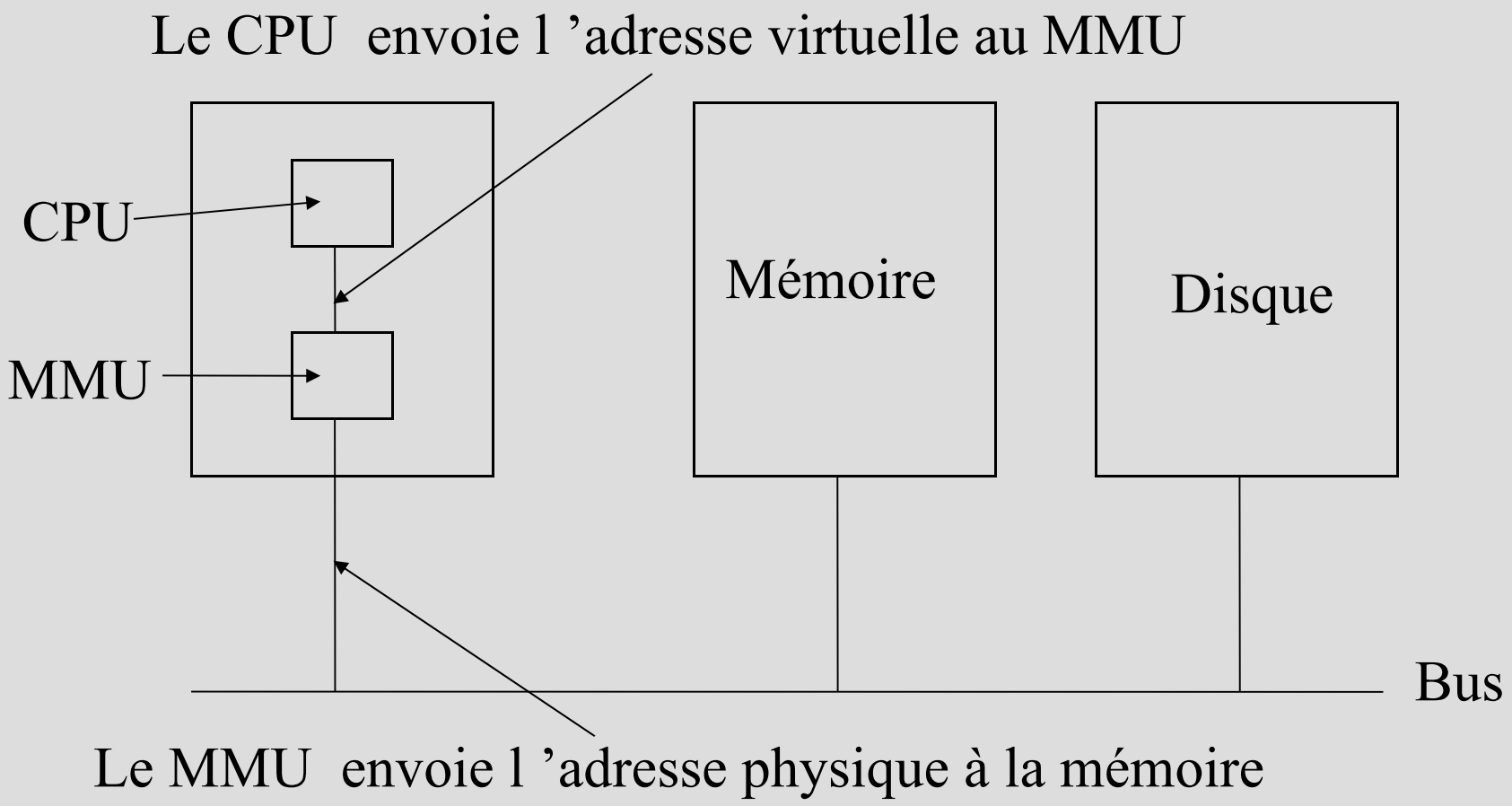
24584

0	**1**	**1**	**0**	0	0	0	0	0	0	0	0	0	1	0	0

Conversion d'adresses virtuelles : Le MMU

- La conversion d'adresse est effectuée par un composant matériel du processeur le MMU : Memory Management Unit.
- Le MMU vérifie si l'adresse virtuelle reçue correspond à une adresse mémoire physique en consultant la table des pages.
- Si c'est le cas, le MMU transmet, sur le bus de la mémoire, l'adresse réelle, sinon il y a un **défaut de page.**
- Un défaut de page provoque un déroutement dont le rôle est de ramener à partir du disque la page manquante référencée.

Conversion d'adresses virtuelles : Le MMU



Algorithmes de remplacement des pages

- A la suite d ’un défaut de page, le système d ’exploitation doit ramener en mémoire la page manquante à partir du disque.
- S’il n ’y a pas de cases libres en mémoire, il doit retirer une page de la mémoire pour la remplacer par celle demandée.
- Si la page à retirer à été modifiée depuis son chargement en mémoire, il faut la récrire sur le disque.
- Les algorithmes mémorisent les références passées aux pages. Le choix de la page à retirer dépend des références passées. Il existe différents algorithmes :
  - l’algorithme optimal de Belady
  - La non récemment utilisé (NRU)
  - l’algorithme FIFO
  - l’algorithme de l’horloge
  - La moins récemment utilisé (LRU)

Remplacement de pages optimal

- Consiste à retirer la page qui sera référencée le plus tard possible dans le futur.
- Cette stratégie est impossible à mettre en œuvre car il est difficile de prévoir les références futures d ’un programme.
- Il est utilisé comme référence pour les autres stratégies car il minimise le nombre de défauts de page
- Nombre d ’accès : 20 et nombre de défauts de pages : 9 (45%)

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	**2**	2	2	2	2	2	2	2	2	2	2	2	2	2	**7**	7	7
	0	0	0	0	0	0	**4**	4	4	**0**	0	0	0	0	0	0	0	0	0
		1	1	1	**3**	3	3	3	3	3	3	3	**1**	1	1	1	1	1	1

Remplacement de pages FIFO

- Il mémorise dans une file FIFO (premier entré, premier sorti) les pages présentes en mémoire. Lorsqu'un défaut de page se produit il retire la plus ancienne, c'est à dire celle qui se trouve en tête de file

- Cette stratégie n 'est pas très bonne car elle n 'est pas fondée réellement sur l 'utilisation de la page

- Anomalie de l 'algorithme: dans certains exemples, si on augmente le nombre de cadres, on peut augmenter le nombre de défauts de page au lieu de le diminuer.

- Nombre d 'accès : 20 et nombre de défauts de pages : 15 (75%)

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	**2**	2	2	2	**4**	4	4	**0**	0	0	0	0	0	0	**7**	7	7
	0	0	0	0	**3**	3	3	**2**	2	2	2	2	**1**	1	1	1	1	**0**	0
		1	1	1	1	**0**	0	0	**3**	3	3	3	3	**2**	2	2	2	2	**1**

Anomalie de l’algorithme FIFO

7	0	1	2	7	0	3	7	0	1	2	3	7	0	1	2	3	2	4	3
7	7	7	**2**	2	2	**3**	3	3	3	3	3	**7**	7	7	**2**	2	2	2	2
	0	0	0	**7**	7	7	7	7	**1**	1	1	1	**0**	0	0	**3**	3	3	3
		1	1	1	**0**	0	0	0	0	**2**	2	2	2	**1**	1	1	1	**4**	4

Nombre d ’accès : 20 et nombre de fautes 15 (75 %)

7	0	1	2	7	0	3	7	0	1	2	3	7	0	1	2	3	2	4	3
7	7	7	7	7	7	**3**	3	3	3	**2**	2	2	2	**1**	1	1	1	1	1
	0	0	0	0	0	0	**7**	7	7	7	**3**	3	3	3	**2**	2	2	2	2
		1	1	1	1	1	1	**0**	0	0	0	**7**	7	7	7	**3**	3	3	3
			2	2	2	2	2	2	**1**	1	1	1	**0**	0	0	0	0	**4**	4

Nombre d ’accès : 20 et nombre de fautes 16 (80 %)

Remplacement de pages LRU

- Consiste à retirer la page la moins récemment utilisée
- Cet algorithme est basé sur le fait qu'une page a tendance à être réutilisée dans un futur proche si elle vient être utilisée
- Nombre d'accès : 20 et nombre de défauts de pages : 12 (60%)

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	**2**	2	2	2	**4**	4	4	**0**	0	0	**1**	1	1	1	1	1	1
	0	0	0	0	0	0	0	0	**3**	3	3	3	3	3	**0**	0	0	0	0
		1	1	1	**3**	3	3	**2**	2	2	2	2	2	2	2	2	**7**	7	7

Comment implanter l'algorithme LRU

- Il faut mémoriser,pour chaque page en mémoire, la date de la dernière référence.
- On associe un registre de n bits à chaque page pour simuler le vieillissement de la page.
- A chaque étape, si la page est présente, on met à 1 le bit de poids le plus fort du registre qui est différent de 1.
- Lorsqu'on veut expulser une page on prend la page dont la valeur du registre est la plus faible.

L’algorithme de l’horloge ou de la seconde chance

- Cet algorithme est une approximation du LRU
- Les pages sont mémorisées dans une liste circulaire en forme d ’horloge.
- Un indicateur pointe sur la page la plus ancienne.
- Lorsqu ’un défaut de page se produit, les pages sont examinées une par une en commençant par celle pointée par l’indicateur
- La première page rencontrée ayant son bit de référence à 0 est remplacée. Le bit R de la page ajoutée est à 1.
- Si le bit R d’une page examinée est différent de 0, il est mis à 0.

L’algorithme de l’horloge : exemple

- Nombre d ’accès : 20 et nombre de défauts de pages : 14 (70%)

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	**2**	2	2	2	**4**	4	4	4	**3**	3	3	3	**0**	0	0	0	0
	0	0	0	0	0	0	0	**2**	2	2	2	2	**1**	1	1	1	**7**	7	7
		1	1	1	**3**	3	3	3	3	**0**	0	0	0	**2**	2	2	2	2	**1**

Cas de Linux et de Windows

Le voleur de pages (processus démon qui exécute l'algorithme de remplacement des pages) est réveillé périodiquement pour voir si le nombre de cases libres en mémoire est au moins égal à un seuil « min »

Windows 10 alloue jusqu'à 17408 Mo soit 17,4 Go comme mémoire virtuelle.

Windows alloue un minimum de 32 Mo par processus.

La gestion des pages de windows est équivalente à celle de Linux

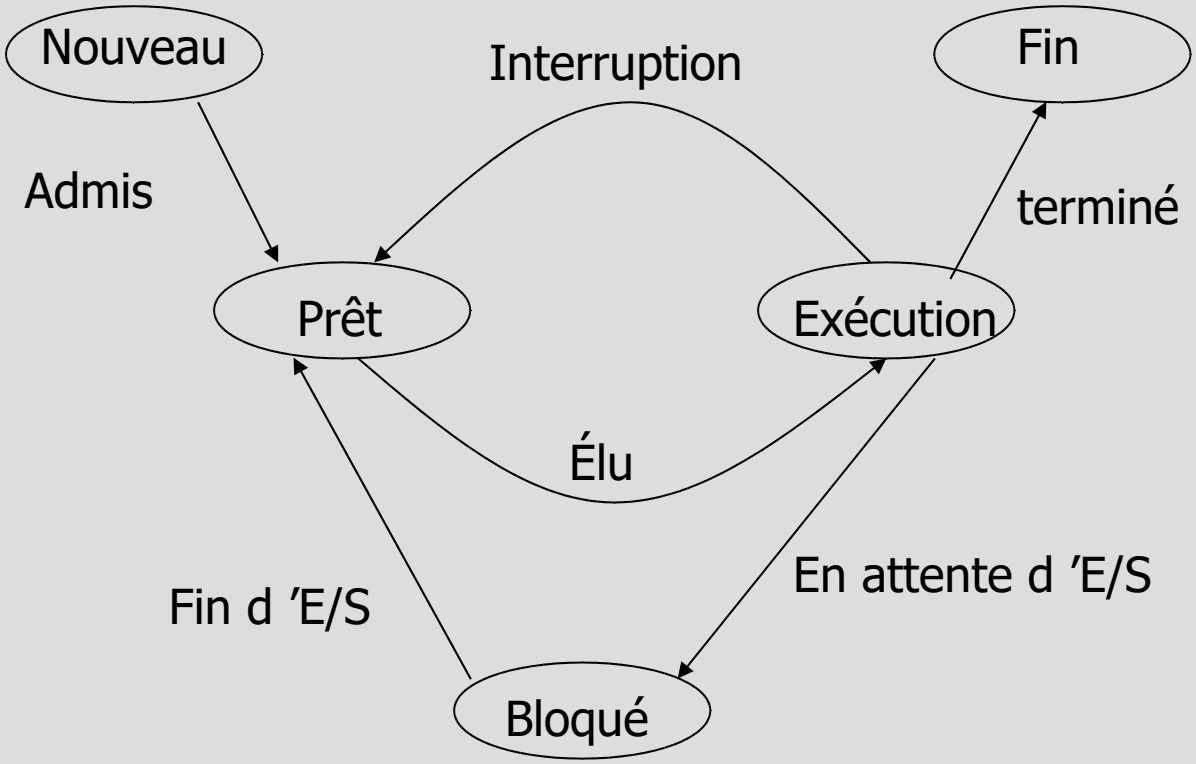
Les processus

- Les processus peuvent se synchroniser et communiquer entre eux.
- Les processus doivent partager les ressources communes de l'ordinateur (unité centrale, mémoire, périphériques, fichiers, etc.).
- Le noyau est chargé du bon déroulement des processus, il doit veiller à ce qu'un processus n'accède pas, sauf autorisation, à ses données en mémoire ou sur disque.
- Le noyau doit répartir le temps d'exécution entre les processus et doit décider du prochain processus à faire passer en unité centrale suivant le principe dit « d'ordonnancement ».

Les états d’un processus

- Lorsqu’un processus s’exécute, il change d’état. Il peut se trouver dans l’un des trois états suivants :
  - Élu : en cours d’exécution (il s’exécute en unité centrale : UC)
  - Prêt : en attente du processeur (il a toutes ses ressources sauf l’UC)
  - Bloqué : en attente d’un événement (il lui manque une (ou plusieurs) ressource pour continuer

Les processus



Synchronisation des processus

- Dans un système d'exploitation plusieurs processus s'exécutent en pseudo-parallèle ou en parallèle et partages des objets (mémoires, imprimantes, …)
- Le partage d'objets sans précaution particulière peut conduire à des résultats imprévisibles. L'état final dépend de l'ordonnancement des processus.
- Lorsqu'un processus modifie un objet partagé, les autres processus ne doivent pas ni la lire, ni la modifier jusqu'à ce que le processus ait terminé de la modifier.
- Autrement dit, l'accès à l'objet doit se faire en **exclusion mutuelle**.

Problème de synchronisation des processus : exemple

- Considérons deux clients A et B d'une banque qui partagent un même compte. Le solde du compte est de 1000$. Supposons qu'en même temps, les deux clients lancent chacun une opération. Le client A demande un retrait de 1000$ alors que le client B veut faire un dépôt de 100$
- Les deux processus exécutant les requêtes des clients partagent la variable « solde »

- Requête de A :

  si (solde >=1000)

  solde = solde -1000;

  sinon erreur

  Requête de B

  solde = solde +100;

- les deux requêtes peuvent être satisfaites et le solde peut passer à 1100$

Ressources critiques : définition

- Il faut empêcher l'utilisation simultanée de la variable commune « solde ». Lorsqu'un processus exécute la séquence d'instructions l'autre doit attendre jusqu'à ce que le premier ait terminé cette séquence.
- Ressource critique : un seul processus peut y accéder à la fois
- Exclusion mutuelle : 2 processus voulant accéder à une ressource critique sont en exclusion mutuelle : l'un ou l'autre, mais pas les deux

Sections critiques : définition

- Section critique : suite d'instructions qui opèrent sur un ou plusieurs objets partagés et qui nécessite une utilisation exclusive des objets partagés.
- Chaque processus à ses propres sections critiques
- Avant d'entamer l'exécution d'une section critique, un processus doit s'assurer de l'utilisation exclusif des objets manipulés par la section critique

Gestion de l'exclusion mutuelle : Sémaphores

- Un sémaphore est un compteur entier qui désigne le nombre d'autorisations d'accès disponibles.
- Chaque sémaphore à un nom et une valeur initiale.
- Les sémaphores sont manipulés au moyen d'opérations :
  - P (S) : décrémente la valeur du sémaphore S si ce dernièr est supérieur à 0. Sinon le processus appelant est mis en attente.
  - V (S) : incrémente la valeur du sémaphore S, si aucun processus n'est bloqué par l'opération P(S). Sinon, l'un d'entre eux sera choisi et redeviendra prêt

Exclusion mutuelle des sémaphores

- Semaphore S = 1

```
Processus P0
Tant que pas fini
{       P(S)
        section_critique_P0
        V(S)
}
```

```
Processus P1
Tant que pas fini
{       P(S)
        section_critique_P1
        V(S)
}
```

P(S) : S=S-1 si S>0
sinon attente

V(S) : S=S+1

- Les sémaphores permettent d'assurer l'exclusion mutuelle

Les sémaphores sous Linux

- Les sémaphores sont implantés dans la librairie « sémaphore.h ».
- Le type est désigné par le mot : **sem_**t.
- Sem_Init : permet l'initialisation d'un sémaphore.
- Sem_destroy : suppression d'un sémaphore.
- Sem_Wait : est équivalent à l'opération P.
- Sem_post : est équivalent à l'opération V.
- Sem_getvalue : retourne la valeur courante du sémaphore.

Problème des philosophes

- Cinq philosophes sont assis autour d'une table. Sur la table, il y a alternativement 5 assiettes, 5 baguettes et un plat de riz.
- Un philosophe passe son temps à manger et à penser.
- Pour manger son plat de riz, un philosophe a besoin de deux baguettes qui sont de part et d'autre de son assiette.

Problème des philosophes : solution

- Les baguettes sont les objets partagés. L'accès et l'utilisation d'une baguette doit se faire en exclusion mutuelle.
- Le problème peut être évité, en ajoutant un tableau de 5 sémaphores (Un sémaphore pour chaque baguette). Les sémaphores sont initialisés à 1.
- Lorsqu'un philosophe i ne parvient pas à prendre une baguette, il se met en attente P(S[i]))
- Lorsqu'un philosophe termine de manger, il vérifie si ses voisins sont en attente. Si c'est le cas, il réveille les voisins qui peuvent manger en appelant l'opération V.

Problème des philosophes : solution

- Si tous les philosophes prennent en même temps une baguette, chacun d'entre eux ne pourra prendre l'autre baguette (situation d'interblocage).
- Pour éviter cette situation, un philosophe ne prend jamais une seule baguette.
- Pour cela, on utilise un sémaphore « plat » qui va gérer le nombre de philosophes qui peuvent accéder au plat.

Le processus Philosophe

Penser ( ); /section non critique/
P(plat) /début section critique/

P(S[i])

prendre_baguette (i) ;
P(S[(i+1)%5])

prendre_baguette ((i+1)%5) ;

manger ( );

poser_baguette (i) ;
V(S[i])

poser_baguette ((i+1)%5) ;
V(S[(i+1)%5])

– V(plat) / fin section critique /

Problème de la bibliothèque

- Une bibliothèque contient uniquement cinq exemplaires d’un ouvrage. Ces exemplaires sont régulièrement d’une part lus par des lecteurs, d’autre part modifiés par un écrivain à partir de l’original en sa possession.

- Ce problème modélise les accès à une base de données. Un ensemble de processus tente constamment d'accéder à la base de données soit pour écrire, soit pour lire des informations.

Problème de la bibliothèque

- Pour assurer une certaine cohérence des données de la base, il faut interdire l'accès (en lecture et en écriture) à tous les processus, si un processus est en train de modifier la base (accède à la base en mode écriture). Par contre, plusieurs processus peuvent accéder à la base, en même temps, en mode lecture.

- Les rédacteurs représentent les processus qui demandent des accès en écriture à la base de données. Les lecteurs représentent les processus qui demandent des accès en lecture à la base de données.

Problème de la bibliothèque

Algorithme de l'écrivain :

```
Tant que vrai faire
    Modifier l'original
    Modifier les 5 exemplaires
Fin Tant que
```

Algorithme d'un lecteur :

```
Tant que vrai faire
    Consulter un des 5 exemplaires
    Sortir;
Fin Tant que
```

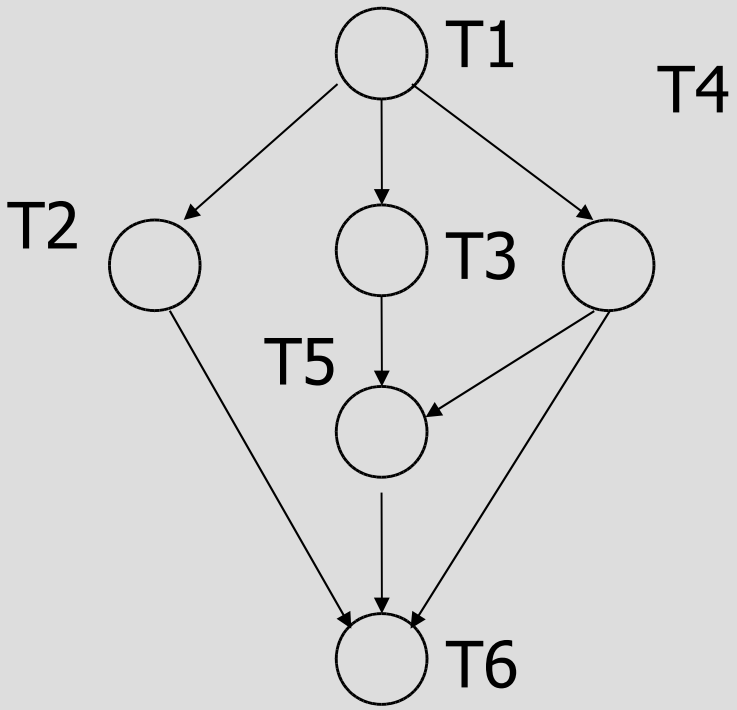
Synchronisation des processus avec des sémaphores

- Une autre application des sémaphores consiste à synchroniser des processus qui collaborent pour réaliser une tâche suivant un ordonnancement bien défini.

Processus P0	Processus P1
…..	….
P(S) Puis-je ?	V(S) vas-y
…..	…..

Les sémaphores : une autre application

- Implanter le graphe de précédence ci contre en utilisant 3 processeurs en parallèle à l'aide de sémaphores.



Les interblocages

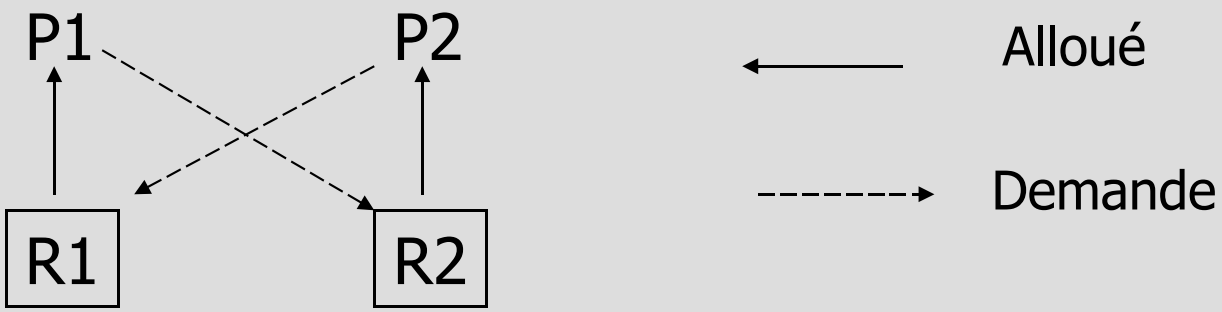
- Introduction
- Qu’est ce qu’un interblocage ?
- Conditions nécessaires pour l’interblocage
- Solutions au problème d’interblocage
  - La détection et la reprise des interblocages
  - L’évitement des interblocages
  - La prévention des interblocages

Interblocages : Introduction

- L'exécution d'un processus nécessite un ensemble de ressources (espace mémoire, espace disque, fichier, périphériques, …) qui lui sont attribués par le système d 'exploitation

- Des problèmes peuvent survenir, si des processus détiennent des ressources et en demandent d'autres qui sont déjà allouées.
- Exemple :
  - Un processus P1 détient une ressource R1 et attend une autre ressource R2 qui est utilisée par un autre processus P2
  - Le processus P2 détient la ressource R2 et attend la ressource R1
  - On a une situation d 'interblocage (P1 attend R2 et P2 attend R1). Les deux processus vont attendre indéfiniment.

Qu ’est ce qu’un interblocage



- Un ensemble de processus est en interblocage si chaque processus attend la libération d ’une ressource allouée à un autre appartenant à l ’ensemble.
- Comme tous les processus sont en attente, aucun ne pourra s ’exécuter et donc libérer les ressources demandées par les autres. Ils attendront tous indéfiniment.

La détection

- Pour détecter les interblocages, le système construit dynamiquement le graphe d'allocation des ressources du système qui indique les attributions et les demandes de ressources

- Le système vérifie s 'il y a des interblocages :

  - A chaque modification du graphe suite à une demande d'une ressource (coûteuse en termes de temps CPU)

  - Périodiquement ou lorsque l'utilisation du processeur est inférieur à un certain seuil (la détection peut être tardive)

Détection : graphe d'allocation des ressources

- Le graphe d'allocation des ressources est un graphe biparti composé de deux types de nœuds et d 'un ensemble d 'arcs :
  - Les processus qui sont représentés par des cercles
  - Les ressources qui sont représentées par des rectangles. Chaque rectangle contient autant de points qu'il y a d'exemplaires de la ressource représentée
  - Un arc orienté d 'une ressource vers un processus signifie que la ressource est allouée au processus
  - Un arc orienté d 'un processus vers une ressource signifie que le processus est bloqué en attente de la ressource
- Ce graphe indique pour chaque processus les ressources qu'il détient ainsi que celles qu'il demande
- La détection est réalisée en réduisant le graphe

Exemple de graphe d'allocation des ressources

- Soient trois processus A, B et C qui utilisent trois ressources R, S et T comme suit

A	B	C
Demande R	Demande S	Demande T
Demande S	Demande T	Demande R
Libère R	Libère S	Libère T
Libère S	Libère T	Libère R

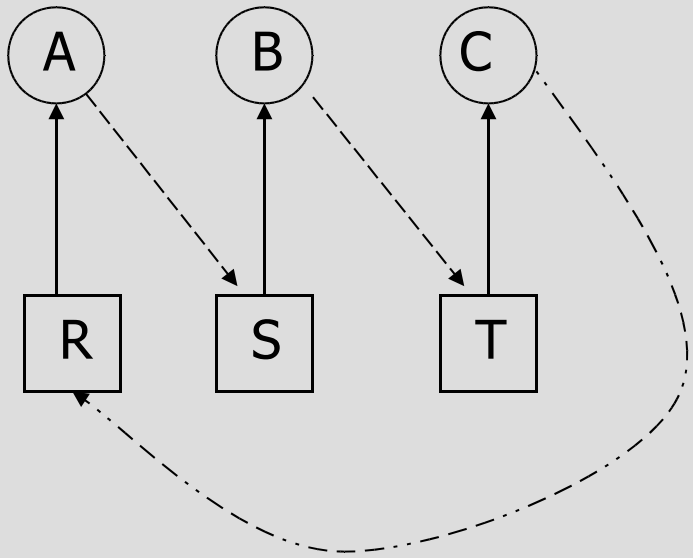
- Si les processus sont exécutés séquentiellement A suivi de B suivi de C, il n 'y a pas d'interblocage

- Supposons que l'exécution des processus est gérée par un Ordonnanceur circulaire. On atteint une situation d 'interblocage

Exemple de graphe d'allocation des ressources

- Si les instructions sont exécutées dans cet ordre :

A Demande R
B Demande S
C Demande T
A Demande S
B Demande T
C Demande R



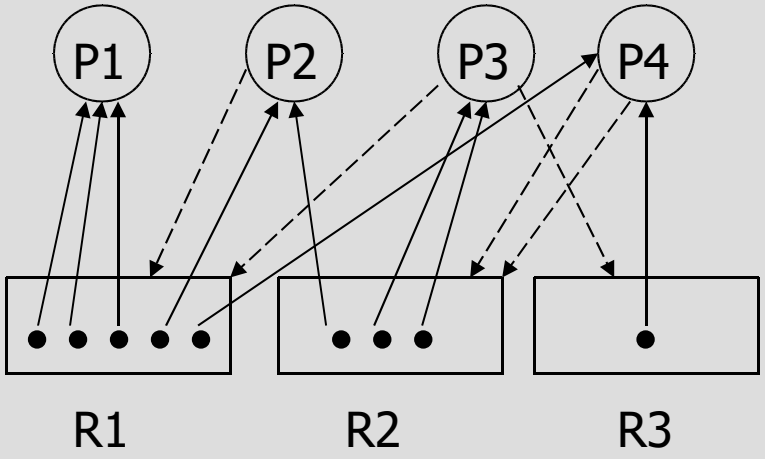
Réduction du graphe d’allocation

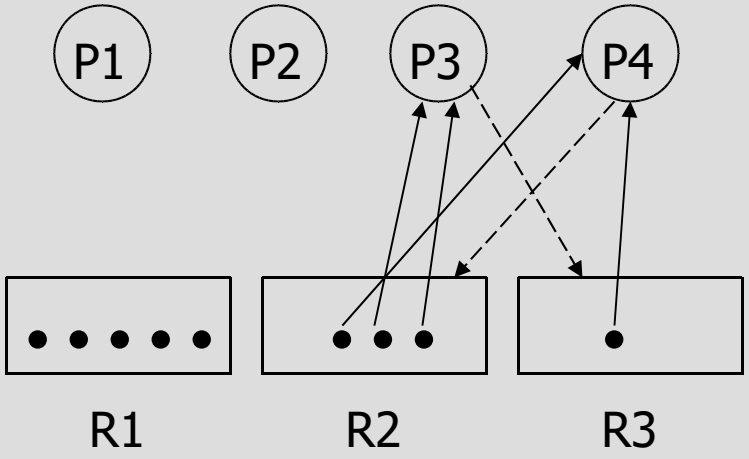
- Pour la réduction d’un graphe d’allocation des ressources, les flèches associées à chaque processus et à chaque ressource doivent être vérifiée :
  - Si une ressource possède seulement des flèches qui sortent (il n ’y a pas de requêtes), on les efface
  - Si un processus possède seulement des flèches qui pointent vers lui, on les effaces
  - Pour chaque flèche de requête, s’il y a une ressource disponible dans le bloc de ressources où la flèche pointe, il faut l’affecter.

Réduction du graphe d'allocation : Exemple

- Le nombre de ressource disponible à l 'instant courant est [0,0,0]

Processus	R1	R2	R3	R1	R2	R3
P1	3	0	0	0	0	0
P2	1	1	0	1	0	0
P3	0	2	0	1	0	1
P4	1	0	1	0	2	0





La reprise

- Lorsque le système détecte un interblocage, il doit le supprimer, ce qui se traduit généralement par la réalisation de l'une des opérations suivantes :
  - Retirer temporairement une ressource à un processus pour l'attribuer à un autre.
  - Restaurer un état antérieur (retour en arrière) et éviter de retomber dans la même situation.
  - Supprimer un ou plusieurs processus

L’évitement des interblocages

- Lorsqu’un processus demande une ressource, le système doit déterminer si l’attribution de la ressource est sûre (s’il existe une séquence d’allocations des ressources qui permet à tous les processus de se terminer)
- Si c’est le cas, il lui attribue la ressource
- Sinon, la ressource n’est pas accordée

Comment déterminer si un état est sûr ou non ?
L'Algorithme du banquier

- Soit n le nombre de processus P1, P2,…, Pn et m le nombre de type de ressources R1, R2,….,Rm qui existent dans un système. L 'algorithme de détection des interblocages utilise les matrices et vecteurs suivants :
  - Matrice « Alloc » des allocations courantes d 'ordre (n x m). L 'élément Alloc[i,j] désigne le nombre de ressources de type Rj détenues par le processus Pi.
  - Matrice « Req » des demandes de ressources d 'ordre (n x m). L 'élément Req[i,j] est le nombre de ressources de type Rj qui manquent au processus Pi pour pouvoir poursuivre son exécution.
  - Vecteur A d 'ordre m. L 'élément A[j] est le nombre de ressources de type Rj disponibles (non attribuées).
  - Vecteur E d 'ordre m. L 'élément E[j] est le nombre total des ressources de type Rj existantes dans le systèmes.

L’Algorithme du banquier

- Rechercher un processus « Pi » non marqué dans la rangé « Pi » de « Req » est inférieur à « A »
- Si un tel processus n ’existe pas alors l ’état est non sûr. L ’algorithme se termine
- Si ce processus existe, ajoutée la rangée « Pi » de « Alloc » à « A », marquer le processus
- Si tous les processus sont marqués alors l ’état est sûr et l ’algorithme se termine, sinon revenir à l‘étape 1

L’ordonnancement des processus

- Introduction
- Objectif et politique de l’ordonnancement
- Ordonnancement non préemptif
- Ordonnancement préemptif
- Études du cas de Linux

L’ordonnancement des processus : introduction

- Tout logiciel d’un ordinateur peut être vu comme un ensemble de processus dont l’exécution est gérée par un processus particulier : l’Ordonnanceur (scheduler en anglais)

- L’Ordonnanceur :
  - décide de l’ordre et du moment d’admission des processus en attente.
  - gère l’allocation du processeur aux différents processus prêts.

- L’exécution d’un processus est une séquence alternée de calculs CPU et d’attentes d’événements (E/S, attente d’un sémaphore, …)

- L’Ordonnanceur se charge des changements de contexte (table PCB) : par exemple lorsque le processus en cours se bloque ou et suspendu, le processeur est alloué à un autre.

Les objectifs de l'Ordonnanceur

- Maximiser le nombre de processus exécutés par unité de temps.
- Minimiser le temps d'attente de chaque processus.
- Maximiser le temps d'utilisation des processeurs.
- Éviter le problème de famine (attente infinie).
- Favoriser les processus les plus prioritaires.
- Minimiser le nombre des changements de contexte (changement de processus).

Les critères de performances de l'ordonnancement

- Temps de séjour d'un processus : temps entre l'arrivée du processus et sa sortie.
- Temps d'attente d'un processus : somme des périodes que le processus passe à l'état prêt.
- Temps de réponse : temps qui s'écoule entre l'entrée d'un processus et le moment où il commence à être traité par le processeur.
- Capacité de traitement : nombre de processus traités par unité de temps.
- Taux d'utilisation du ou des processeurs.

Politique d’ordonnancement

- Choix du processus à exécuter ?
  - premier arrivé, premier servi.
  - moins récemment utilisé (LRU : Least Recently Used).
  - plus prioritaire (priorités statiques ou dynamiques).
  - plus court temps d’abord.

- Les temps d’allocation du processeur au processus choisi :
  - allocation jusqu’à terminaison ou libération volontaire (ordonnancement non préemptif).
  - temps d’allocation limité fixe ou variable (ordonnancement préemptif).

- Quand appeler l’Ordonnanceur ?
  - le processus en cours se bloque ou cède le processeur.
  - le temps alloué au processus a expiré.
  - un processus plus prioritaire est arrivé.

Ordonnancement non préemptif (sans réquisition)

- Le système d'exploitation choisit le processus à exécuter selon une combinaison des règles suivantes :
  - Premier arrivé, premier servi (FCFS, First-Come First-Served)
  - Plus court d'abord (SPF, Short Process First)
  - Plus prioritaire d'abord
- Il alloue le processeur au processus jusqu'à ce qu'il se termine, se bloque (en attente d'un événement) ou cède le processeur. Il n'y a pas de réquisition du processeur.

Ordonnancement non préemptif FCFS

Processus	Exécution (s)	Arrivée (s)
A	3	0
B	6	1
C	4	4
D	2	6
E	1	7

AAABBBBBBCCCCDDE

Temps de séjour moyen : 7.6 s

Temps moyen d ’attente : 4.4 s

Nombre de changement de contexte : 5

Unités de temps par processus : 3.2 s

Temps moyen d’attente élevé si de longs processus sont exécutés en premier

Ordonnancement non préemptif
SPF

Processus	Exécution (s)	Arrivée (s)
A	3	0
B	6	1
C	4	4
D	2	6
E	1	7

AAABBBBBBEDDCCCC

Temps de séjour moyen : 6.4 s

Temps moyen d ’attente : 3.2 s

Nombre de changement de contexte : 5

Meilleur temps moyen d’attente

Ordonnancement préemptif (avec réquisition)

- Pour s'assurer qu'aucun processus ne s'exécute pendant trop longtemps, les ordinateurs ont une horloge électronique qui génère périodiquement une interruption.

- A chaque interruption d'horloge, le système d'exploitation reprend la main et décide si :
  - le processus courant doit poursuivre son exécution

  ou
  - le processus doit être suspendu pour laisser la place à un autre.

- S'il décide de suspendre l'exécution du processus au profit d'un autre, il doit d'abord sauvegarder l'état des registres du processeur avant de charger dans les registres les données du processus à lancer (commutation de contexte)

Ordonnancement préemptif (avec réquisition)

- Le processeur passe d'un processus à un autre en exécutant chaque processus pendant quelques dizaines ou centaines de millisecondes.

- Le temps d'allocation du processeur au processus est appelé **quantum.** Cette commutation entre processus doit être rapide, c'est à dire un temps nettement inférieur au quantum.

- Le processeur, à un instant donné n'exécute réellement qu'un seul processus. Mais pendant une seconde, le processeur peut exécuter plusieurs processus et il donne l'impression d'une exécution en parallèle (pseudo parallélisme).

Choix de la valeur du quantum

- Un quantum trop petit provoque beaucoup de commutations de processus.
- Un quantum trop élevé augmente le temps de réponse des courtes commandes en mode interactif.
- Le quantum est de quelques millisecondes et était de 1 seconde dans les premières versions d'Unix.

Ordonnancement préemptif SRT (Shortest Remaining Time)

Processus	Exécution (s)	Arrivée (s)
A	3	0
B	6	1
C	4	4
D	2	6
E	1	7

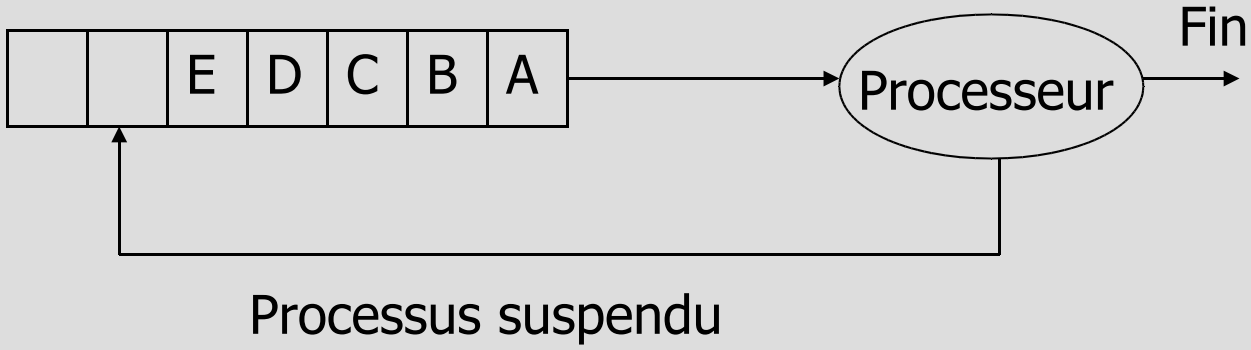
- Lorsqu ’un processus arrive, l’Ordonnanceur compare le temps estimé de son exécution avec celle du processus actuellement en exécution (version préemptive du SPF).
- Si ce temps est plus petit, on exécute immédiatement le nouveau.
- A temps d’exécution identique, on choisit le processus qui vient d’être exécuté.

AAABCCCCEDDBBBBB

Temps de séjour moyen : 5.8 s
Temps moyen d ’attente : 2.6 s
Nombre de changement de contexte : 6

Ordonnancement réaliste : Ordonnancement circulaire (Tourniquet)

- Algorithme ancien, simple et fiable.
- Il mémorise dans une file (FIFO : First in First Out) la liste des processus en attente d'exécution



Ordonnancement circulaire : exemple

Processus	Exécution (s)	Arrivée (s)
A	3	0
B	6	1
C	4	4
D	2	6
E	1	7

ABABACBDCEBDCBCB

Temps de séjour moyen : 8.0 s

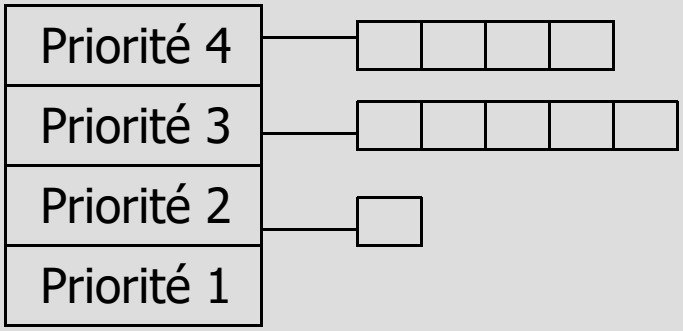
Temps moyen d ’attente : 4.8 s

Nombre de changement de contexte : 16

Quantum = 1 s et temps de commutation = 0

Ordonnancement à priorité avec files multiples

- L ’Ordonnanceur attribue une priorité à chaque processus (statique ou dynamique).
- Les processus de même priorité sont dans une file FIFO.
- Il y a autant de files qu ’il y a de niveaux de priorité.
- Les processus de chaque file sont ordonnancés selon le principe du tourniquet.



Ordonnancement avec files multiples : Exercice

- Un système utilise une file d'exécution. La file est gérée par tourniquet avec une valeur du quantum égale à 1. La priorité initiale de chaque processus (noté de T1 à T5) est donnée dans le tableau ci-après.

- Le système est également doté d'une pagination suivant l'algorithme FIFO. Au cours de son exécution une tâche accède à une des pages numérotées de 1 à 5 (cf tableau). Le système alloue en permanence un espace fixe de trois cases en mémoire. A chaque fois qu'un processus est lancé sa priorité est incrémentée du nombre de défaut de pages qu'il a effectué.

Processus	τi (quanta)	Ti (quanta)	Priorité initiale	Pages
T1	3	0	5	1,2
T2	2	ε	6	2,3
T3	1	1	7	3,4
T4	3	$1+2\varepsilon$	8	4,5
T5	2	$1+\varepsilon$	9	5,6

Problèmes d'équité

- Équité : l'exécution des processus moins prioritaires peut être constamment retardée par l'arrivée de processus plus prioritaires.
- Il peut y avoir une inversion des priorités :
  - Un processus moins prioritaire détient une ressource nécessaire à l'exécution d'un processus plus prioritaire.
  - Le processus moins prioritaire ne peut s'exécuter car il y a constamment des processus actifs plus prioritaires.

- Si un processus prêt reste trop longtemps en attente d'une ressource, il se voit augmenter sa priorité temporairement. Cela permet d'éviter les problèmes de famine et d'inversion de priorité..

Cas de Linux

- L'Ordonnanceur utilise plusieurs files, une priorité est associée à chaque file.
- Les processus prêts qui sont en mémoire, sont répartis dans les files selon leur priorité
- Les priorités des processus s'exécutant en mode utilisateur sont positives ou nulles, alors que celles des processus s'exécutant en mode noyau (ou du super utilisateur) sont négatives (les plus élevées)
- L'Ordonnanceur utilise le principe du tourniquet.
- Le processus élu est alors exécuté pendant au maximum un quantum (environ 100ms). S'il ne se termine pas ou ne se bloque pas au bout du quantum, il est suspendu.

Cas de Linux

- Le processus suspendu est inséré en queue de sa file.
- La commande « nice » permet d’attribuer une priorité plus basse à un processus (>0).
- La commande « renice » permet de changer la priorité d’un processus en cours d’exécution.
- Les priorités des processus prêts sont recalculées périodiquement toutes les secondes en prenant en compte le temps d'utilisation du CPU et le nombre de défauts de page.